Unit-5

Graph

Unit-5

- Basic Terminologies
- Representation of a Graph
  - Adjacency Matrix
  - Adjacency List
- BFS Traversal
- DFS Traversal

Basic Terminologies

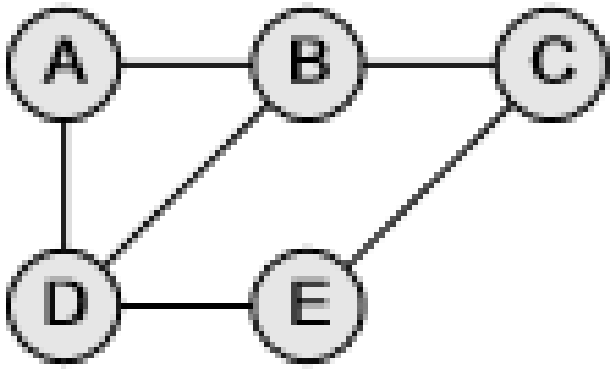
❖ Graph :

- A graph is basically, **"a collection of vertices (also called nodes) and edges that connect these vertices".**
- A graph is often viewed as a generalization of the tree structure, where instead of a having a purely parent-to-child relationship between tree nodes, any kind of complex relationships between the nodes can be represented.
- **A graph G=<V,E> is defined as an ordered set (V,E),where V(G) represents set of vertices and E(G) represents the edges that connect the vertices**.

Basic Terminologies

❖ Graph :

- The figure given shows a graph with V(G) = { A, B, C, D and E} and E(G) = { (A, B), (B, C), (A, D), (B, D), (D, E), (C, E) }. Note that there are 5 vertices or nodes and 6 edges in the graph.



Basic Terminologies

❖ Adjacent Nodes:

- Any two nodes which are connected by an edge in a graph are called Adjacent nodes.

❖ Directed Edge and Undirected Edge:

- In a graph G = (V,E), an edge which is directed from one node to another is called a directed edge, while an edge which has no specific direction is called an undirected edge.

Basic Terminologies

❖ Directed Graph :

- A graph in which every edge is directed is called a directed graph or diagraph.

❖ Undirected Graph :

- A graph in which every edge is undirected is called an undirected graph.

❖ Mixed Graph:

- A graph in which some of the edges are directed and some are undirected is called mixed graph.

Basic Terminologies

❖ Loop or Sling :

- An edge of a graph which joins a node to itself is called a loop.

❖ Tree :

- A tree consists of finite set of elements called nodes, and finite set of directed lines, called branches, that connect the nodes.

Basic Terminologies

❖ Parallel Edges:

- Distinct edges which connect the same end points are called Parallel edges.

Basic Terminologies

❖ Multigraph :

- Any graph which contains some parallel edges is called a multigraph.

❖ Simple graph:

- If a graph does not contain any parallel edges or loop then it is Simple graph.

❖ Multiplicity :

- In graph, number on any edge shows the multiplicity of the edge.
- We may also consider the multiplicity as a weight assigned to an edge.

Basic Terminologies

❖ Weighted Graph:

- A graph in which weights are assigned to every edge is called weighted graph.

❖ Isolated Node :

- In a graph, a node which is not adjacent to any other node is called an isolated graph.

❖ Null Graph :

- A graph containing only isolated nodes is called a null graph.

Basic Terminologies

❖ Cycle Graph:

- A cycle graph is a graph consisting of a single cycle. The cycle graph with n vertices is denoted by C_n.

❖ Acyclic Graph:

- A simple diagraph which does not have any cycle is called acyclic graph.

Basic Terminologies

❖ Path:

- A path of length n from u to v, where n is a positive integer, in an undirected graph is a sequence of edges e_1, e_2, …, e_n of the graph such that $e_1 = \{x_0, x_1\}$, $e_2 = \{x_1, x_2\}$, …, $e_n = \{x_{n-1}, x_n\}$, where $x_0 = u$ and $x_n = v$.

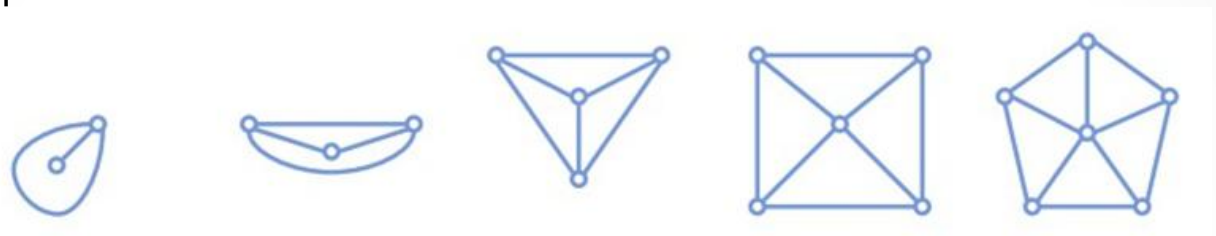
❖ Circuit :

- The path is a circuit if it begins and ends at the same vertex, that is, if u = v.

Basic Terminologies

❖ Wheels:

- The wheel graph W_n is just a cycle graph with an extra vertex in the middle.



❖ Degree :

- Degree of a node u, deg(u), is the total number of edges containing the node u.

❖ Length :

- The no. of edges appearing in the sequence of a path is called the length of the path.

Representation of a Graph

- There are two common ways of storing graphs in computer's memory.

1. Sequential Representation - Adjacency Matrix (Static)
2. Linked Representation - Adjacency List (Dynamic)

Adjacency Matrix

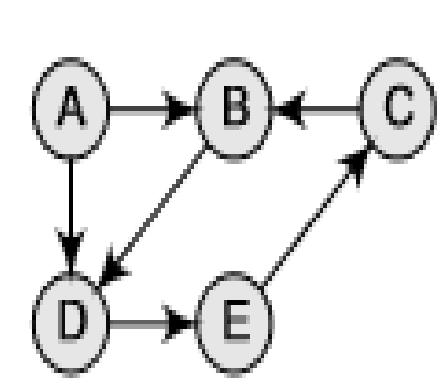
- An adjacency matrix is used to represent which nodes are adjacent to one another. By definition, we have learnt that, two nodes are said to be adjacent if there is an edge connecting them.
- In a directed graph G, if node v is adjacent to node u, then surely there is an edge from u to v. That is, if v is adjacent to u, we can get from u to v by traversing one edge. For any graph G having n nodes, the adjacency matrix will have dimensions of n x n.
- In an adjacency matrix, the rows and columns are labeled by graph vertices. An entry a_{ij} in the adjacency matrix will contain 1, if vertices v_i and v_j are adjacent to each other. However, if the nodes are not adjacent, a_{ij} will be set to zero.

Adjacency Matrix

$$a_{ij} \begin{cases} 1 & [\text{if } v_i \text{ is adjacent to } v_j, \text{ that is there is an edge } (v_i, v_j)]A \\ 0 & [\text{otherwise}] \end{cases}$$

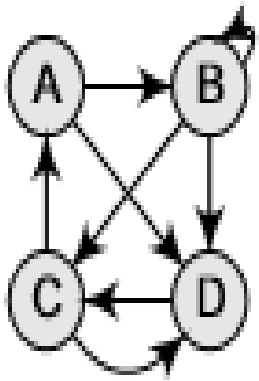
- Since an adjacency matrix contains only 0s and 1s, it is called a **Bit Matrix** or a **Boolean Matrix.**
- The entries in the matrix depend on the ordering of the nodes in G.
- Therefore, a change in the order of nodes will result in a different adjacency matrix.

Adjacency Matrix



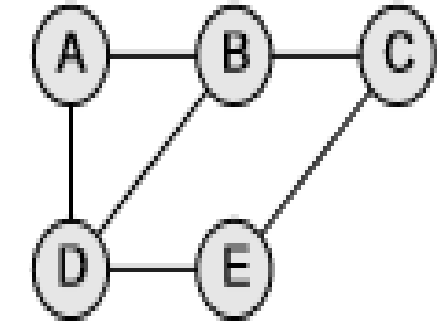
	A	B	C	D	E
A	0	1	0	1	0
B	0	0	0	1	0
C	0	1	0	0	0
D	0	0	0	0	1
E	0	0	1	0	0

(a) Directed graph



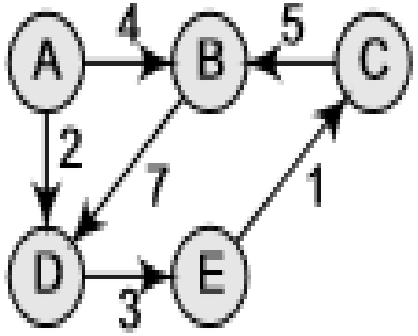
	A	B	C	D
A	0	1	0	1
B	0	1	1	1
C	1	0	0	1
D	0	0	1	0

(b) Directed graph with loop



	A	B	C	D	E
A	0	1	0	1	0
B	1	0	1	1	0
C	0	1	0	0	1
D	1	1	0	0	1
E	0	0	1	1	0

(c) Undirected graph

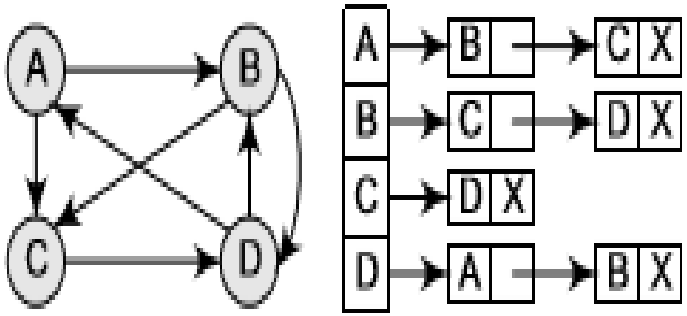


	A	B	C	D	E
A	0	4	0	2	0
B	0	0	0	7	0
C	0	5	0	0	0
D	0	0	0	0	3
E	0	0	1	0	0

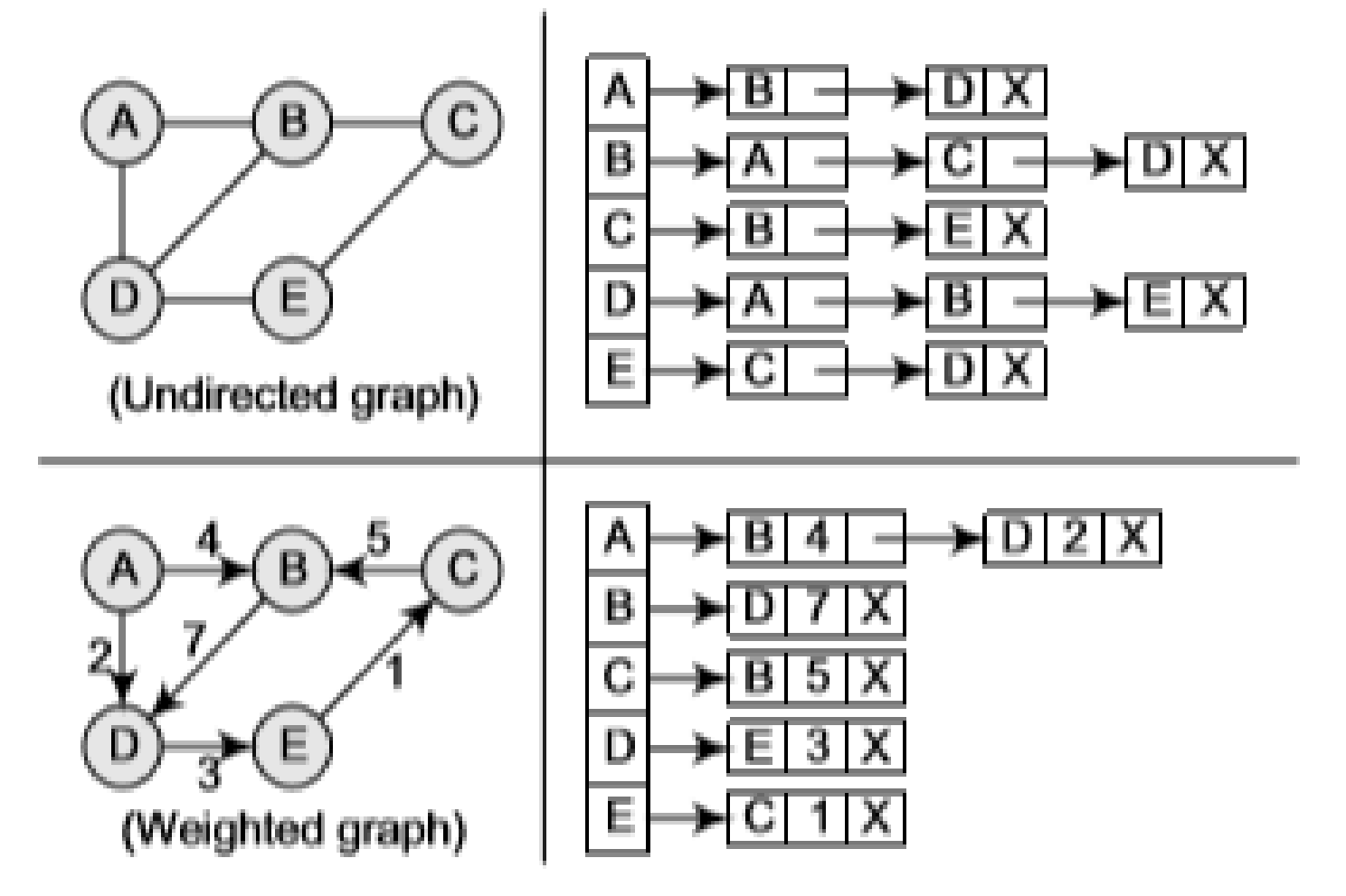
(d) Weighted graph

Adjacency List

- The adjacency list is another way in which graphs can be represented in computer's memory.
- This structure consists of a list of all nodes in G.
- Furthermore, every node is in turn linked to its own list that contains the names of all other nodes that are adjacent to itself.



Adjacency List



BFS

- BFS stands for Breadth First Search is a vertex based technique for finding a shortest path in graph. It uses a Queue data structure which follows first in first out. In BFS, one vertex is selected at a time when it is visited and marked then its adjacent are visited and stored in the queue.

- Breadth First Search (BFS) algorithm traverses a graph in a breadthward motion and uses a queue to remember to get the next vertex to start a search, when a dead end occurs in any iteration.

BFS

- Breadth-first search (BFS) is a graph search algorithm that begins at the root node and explores all
- the neighbouring nodes. Then for each of those nearest nodes, the algorithm (Fig. 13.20) explores
- their unexplored neighbour nodes, and so on, until it finds the goal.

BFS

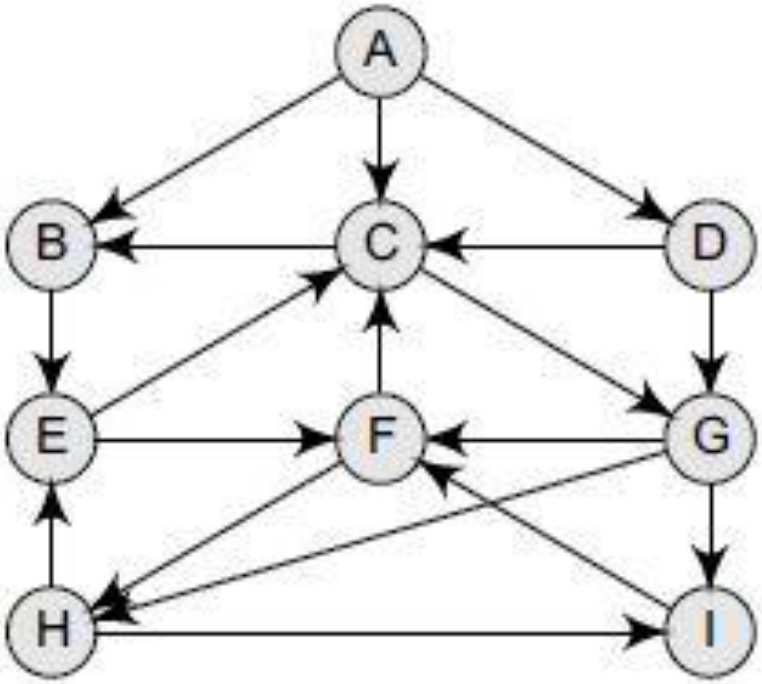
```
Step 1: SET STATUS = 1 (ready state)
        for each node in G
Step 2: Enqueue the starting node A
        and set its STATUS = 2
        (waiting state)
Step 3: Repeat Steps 4 and 5 until
        QUEUE is empty
Step 4: Dequeue a node N. Process it
        and set its STATUS = 3
        (processed state).
Step 5: Enqueue all the neighbours of
        N that are in the ready state
        (whose STATUS = 1) and set
        their STATUS = 2
        (waiting state)
        [END OF LOOP]
Step 6: EXIT
```

Figure 13.20 Algorithm for breadth-first search

BFS

- That is, we start examining the node A and then all the neighbours of A are examined. In the next step, we examine the neighbours of neighbours of A, so on and so forth. This means that we need to track the neighbours of the node and guarantee that every node in the graph is processed and no node is processed more than once. This is accomplished by using a queue that will hold the nodes that are waiting for further processing and a variable STATUS to represent the current state of the node.

BFS



Adjacency lists

A: B, C, D
B: E
C: B, G
D: C, G
E: C, F
F: C, H
G: F, H, I
H: E, I
I: F

Figure 13.21 Graph G and its adjacency list

BFS

- Consider the graph G given in Fig. 13.21. The adjacency list of G is also given. Assume that G represents the daily flights between different cities and we want to fly from city A to I with minimum stops. That is, find the minimum path P from A to I given that every edge has a length of 1.
- The minimum path P can be found by applying the breadth-first search algorithm that begins at city A and
- ends when I is encountered. During the execution of the algorithm, we use two arrays:
- QUEUE and ORIG.
- While QUEUE is used to hold the nodes that have to be processed, ORIG is used to keep track of the origin of each edge. Initially, FRONT = REAR = –1.

BFS

(a) Add A to QUEUE and add NULL to ORIG.

FRONT = 0	QUEUE = A
REAR = 0	ORIG = \0

(b) Dequeue a node by setting FRONT = FRONT + 1 (remove the FRONT element of QUEUE) and enqueue the neighbours of A. Also, add A as the ORIG of its neighbours.

FRONT = 1	QUEUE = A	B	C	D
REAR = 3	ORIG = \0	A	A	A

(c) Dequeue a node by setting FRONT = FRONT + 1 and enqueue the neighbours of B. Also, add B as the ORIG of its neighbours.

FRONT = 2	QUEUE = A	B	C	D	E
REAR = 4	ORIG = \0	A	A	A	B

(d) Dequeue a node by setting FRONT = FRONT + 1 and enqueue the neighbours of C. Also, add C as the ORIG of its neighbours. Note that C has two neighbours B and G. Since B has already been added to the queue and it is not in the Ready state, we will not add B and only add G.

FRONT = 3	QUEUE = A	B	C	D	E	G
REAR = 5	ORIG = \0	A	A	A	B	C

BFS

(e) Dequeue a node by setting FRONT = FRONT + 1 and enqueue the neighbours of D. Also, add D as the ORIG of its neighbours. Note that D has two neighbours C and G. Since both of them have already been added to the queue and they are not in the Ready state, we will not add them again.

FRONT = 4	QUEUE = A	B	C	D	E	G
REAR = 5	ORIG = \0	A	A	A	B	C

(f) Dequeue a node by setting FRONT = FRONT + 1 and enqueue the neighbours of E. Also, add E as the ORIG of its neighbours. Note that E has two neighbours C and F. Since C has already been added to the queue and it is not in the Ready state, we will not add C and add only F.

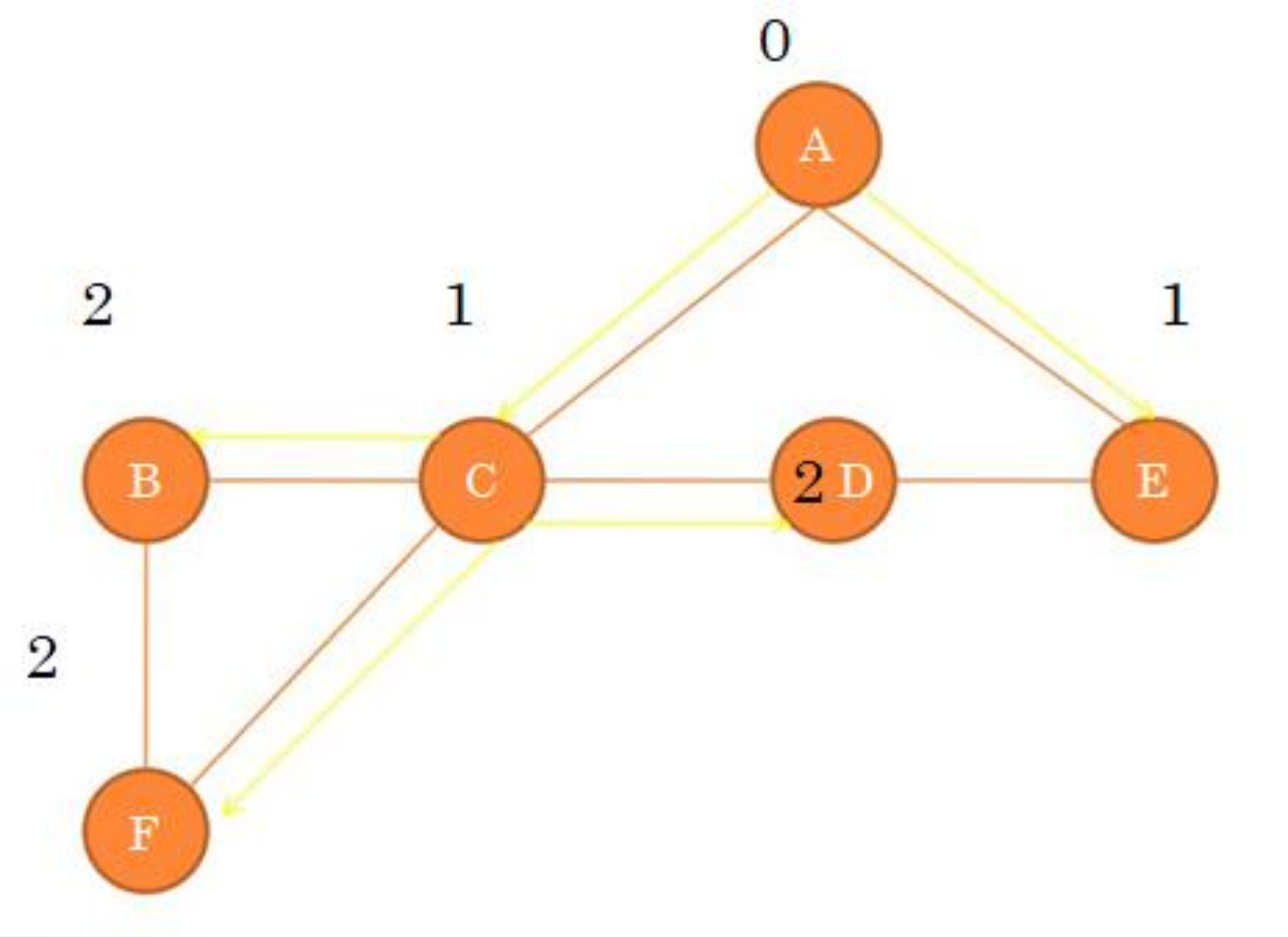
FRONT = 5	QUEUE = A	B	C	D	E	G	F
REAR = 6	ORIG = \0	A	A	A	B	C	E

(g) Dequeue a node by setting FRONT = FRONT + 1 and enqueue the neighbours of G. Also, add G as the ORIG of its neighbours. Note that G has three neighbours F, H, and I.

FRONT = 6	QUEUE = A	B	C	D	E	G	F	H	I
REAR = 9	ORIG = \0	A	A	A	B	C	E	G	G

Since F has already been added to the queue, we will only add H and I. As I is our final destination, we stop the execution of this algorithm as soon as it is encountered and added to the QUEUE. Now, backtrack from I using ORIG to find the minimum path P. Thus, we have P as A –> C –> G –> I.

BFS



BFS TRAVERSAL OF GRAPH : A C E B F D

DFS

- Depth First Search (DFS) algorithm traverses a graph in a depthward motion and uses a stack to remember to get the next vertex to start a search, when a dead end occurs in any iteration.

- The depth-first search algorithm (Fig. 13.22) progresses by expanding the starting node of G and then going deeper and deeper until the goal node is found, or until a node that has no children is encountered. When a dead-end is reached, the algorithm backtracks, returning to the most recent node that has not been completely explored.

DFS

```
Step 1: SET STATUS = 1 (ready state) for each node in G
Step 2: Push the starting node A on the stack and set
        its STATUS = 2 (waiting state)
Step 3: Repeat Steps 4 and 5 until STACK is empty
Step 4:   Pop the top node N. Process it and set its
          STATUS = 3 (processed state)
Step 5:   Push on the stack all the neighbours of N that
          are in the ready state (whose STATUS = 1) and
          set their STATUS = 2 (waiting state)
        [END OF LOOP]
Step 6: EXIT
```

Figure 13.22 Algorithm for depth-first search

DFS

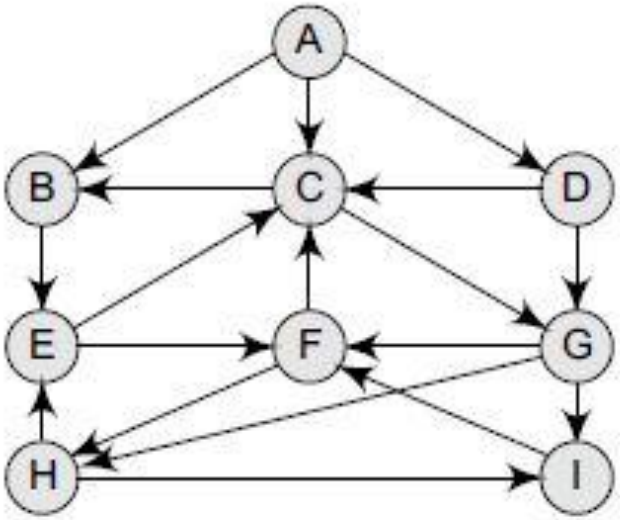
- In other words, depth-first search begins at a starting node A which becomes the current node.Then, it examines each node N along a path P which begins at A. That is, we process a neighbour of A, then a neighbour of neighbour of A, and so on. During the execution of the algorithm, if we reach a path that has a node N that has already been processed, then we backtrack to the current node. Otherwise, the unvisited (unprocessed) node becomes the current node.

DFS

- The algorithm proceeds like this until we reach a dead-end (end of path P). On reaching the deadend, we backtrack to find another path P¢. The algorithm terminates when backtracking leads back to the starting node A. In this algorithm, edges that lead to a new vertex are called discovery edges and edges that lead to an already visited vertex are called back edges.

DFS

- Consider the graph G given in Fig. 13.23. The adjacency list of G is also given. Suppose we want to print all the nodes that can be reached from the node H (including H itself). One alternative is to use a depth-first search of G starting at node H.



Adjacency lists

A: B, C, D
B: E
C: B, G
D: C, G
E: C, F
F: C, H
G: F, H, I
H: E, I
I: F

Figure 13.23 Graph G and its adjacency list

DFS

Solution

(a) Push H onto the stack.

STACK: H

(b) Pop and print the top element of the STACK, that is, H. Push all the neighbours of H onto the stack that are in the ready state. The STACK now becomes

PRINT: H

STACK: E, I

(c) Pop and print the top element of the STACK, that is, I. Push all the neighbours of I onto the stack that are in the ready state. The STACK now becomes

PRINT: I

STACK: E, F

(d) Pop and print the top element of the STACK, that is, F. Push all the neighbours of F onto the stack that are in the ready state. (Note F has two neighbours, C and H. But only C will be added, as H is not in the ready state.) The STACK now becomes

PRINT: F

STACK: E, C

DFS

(e) Pop and print the top element of the STACK, that is, C. Push all the neighbours of C onto the stack that are in the ready state. The STACK now becomes

PRINT: C | STACK: E, B, G

(f) Pop and print the top element of the STACK, that is, G. Push all the neighbours of G onto the stack that are in the ready state. Since there are no neighbours of G that are in the ready state, no push operation is performed. The STACK now becomes

PRINT: G | STACK: E, B

(g) Pop and print the top element of the STACK, that is, B. Push all the neighbours of B onto the stack that are in the ready state. Since there are no neighbours of B that are in the ready state, no push operation is performed. The STACK now becomes

PRINT: B | STACK: E

(h) Pop and print the top element of the STACK, that is, E. Push all the neighbours of E onto the stack that are in the ready state. Since there are no neighbours of E that are in the ready state, no push operation is performed. The STACK now becomes empty.

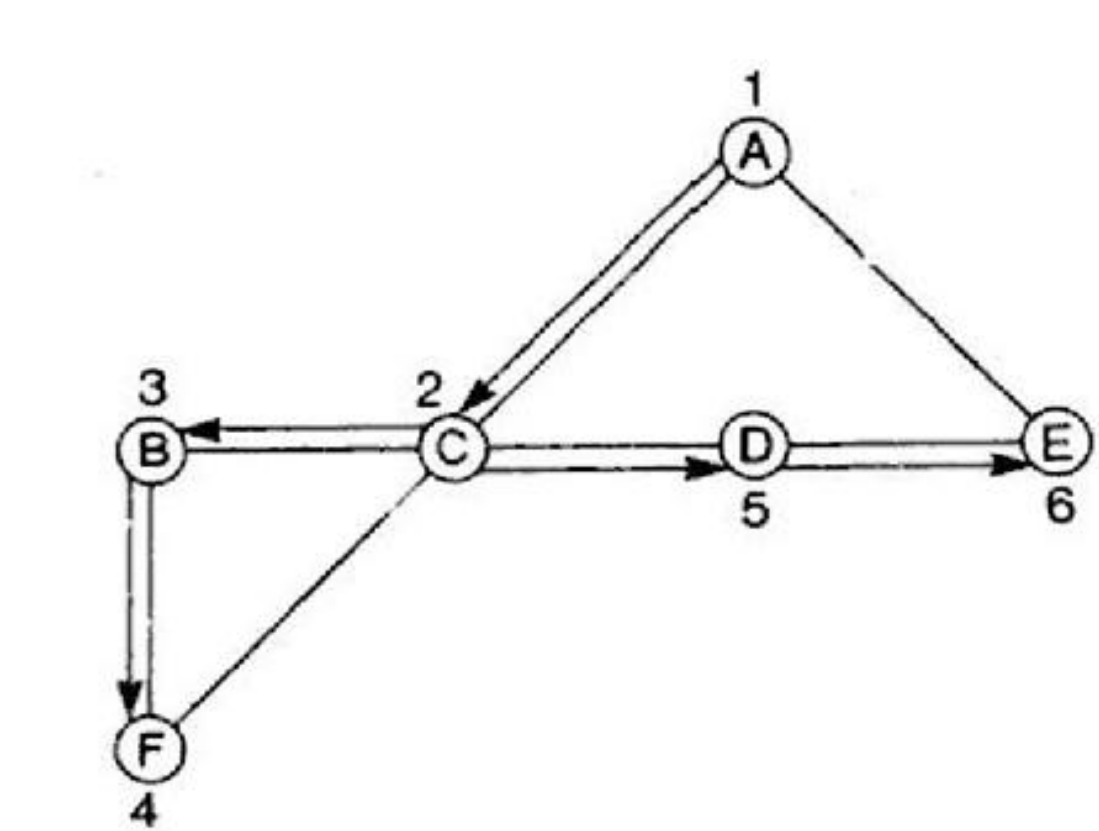
PRINT: E | STACK:

Since the STACK is now empty, the depth-first search of G starting at node H is complete and the nodes which were printed are:

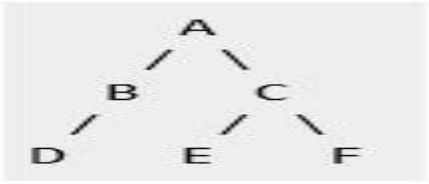
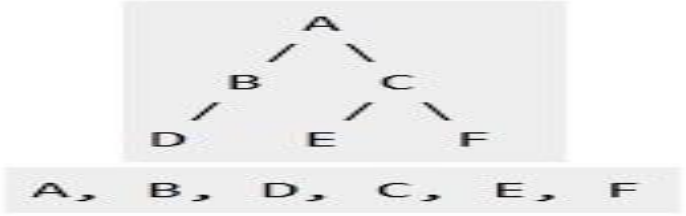
```
H, I, F, C, G, B, E
```

These are the nodes which are reachable from the node H.

DFS



DFS TRAVERSAL OF GRAPH : A C B F D E

BFS	DFS
BFS Stands for "**Breadth First Search**".	**DFS** stands for "**Depth First Search**".
BFS starts traversal from the root node and then explore the search in the level by level manner i.e. as close as possible from the root node.	DFS starts the traversal from the root node and explore the search as far as possible from the root node i.e. depth wise.
Breadth First Search can be done with the help of **queue** i.e. **FIFO** implementation.	Depth First Search can be done with the help of **Stack** i.e. **LIFO** implementations.
This algorithm works in single stage. The visited vertices are removed from the queue and then displayed at once.	This algorithm works in two stages – in the first stage the visited vertices are pushed onto the stack and later on when there is no vertex further to visit those are popped-off.
BFS is **slower** than DFS.	DFS is more **faster** than BFS.
BFS requires **more** memory compare to DFS.	DFS require **less** memory compare to BFS.
BFS is useful in finding shortest path.BFS can be used to find the shortest distance between some starting node and the remaining nodes of the graph.	DFS in not so useful in finding shortest path. It is used to perform a traversal of a general graph and the idea of DFS is to make a path as long as possible, and then go back (**backtrack**) to add branches also as long as possible.
Ex. 	Ex. 

Thank You…!!!!